嵌入式 RTOS 与 I2C 传感器实验报告

实验题目： uC/OS-III 在 STM32F103 上的移植与多任务设计；基于 I2C 的 AHT20 温湿度采集

日期： 2026 年 11 月

1 实验目的

1. 理解嵌入式实时操作系统的基本概念（任务、优先级、调度、节拍、上下文切换）。
2. 掌握 uC/OS-III 在 Cortex-M3（STM32F103C8T6）上的移植方法，能独立完成工程搭建、配置裁剪和中断对接。
3. 掌握多任务程序的划分与实现：三个任务分别以 1 s、3 s、2 s 为周期运行，验证抢占式调度。
4. 理解 I2C 总线协议，掌握软件 I2C 与硬件 I2C 的区别。
5. 能阅读 AHT20 数据手册，完成初始化、触发测量、数据解析，并通过串口输出。
6. 会用示波器观察 I2C 时序，能对照手册说明地址、命令和时序参数。
7. 把裸机采集程序改造为 RTOS 任务，理解阻塞延时与任务让出的区别。

2 实验环境

类别	内容
主控	STM32F103C8T6 最小系统板（Blue Pill），64 KB Flash / 20 KB SRAM
晶振	外部 8 MHz，PLL ×9 → 72 MHz
传感器	AHT20 温湿度模块（I2C，7 位地址 0x38）
调试器	ST-Link V2（SWD）
串口	USB-TTL（CH340），USART1，PA9/PA10，115200-8-N-1
IDE	Keil MDK 5.37，ARM Compiler 6（V6.18，armclang）
库	STM32Cube HAL（STM32F1xx HAL Driver）
RTOS	uC/OS-III V3.08.02（Apache-2.0）+ uC-CPU + uC-LIB
上位机	Win10 + 串口助手（XCOM / SSCOM，UTF-8 编码）
仪器	示波器（或 Keil 逻辑分析仪）

3 任务一：uC/OS-III 的移植与三任务设计

3.1 RTOS 与 uC/OS-III

裸机程序是“一个大循环 + 中断”，所有功能串行执行，长延时（如 `HAL_Delay(2000)`）期间 CPU 空转，其他功能得不到响应。RTOS 把功能拆成多个独立任务，由内核按优先级调度，用系统节拍（Tick）驱动延时和超时，延时期间 CPU 去运行其他任务。

uC/OS-III 是一个可裁剪、可剥夺的实时内核，主要特性：

- 任务优先级 0～63（可配置），数值越小优先级越高；
- 每个任务有独立的栈和 TCB（任务控制块）；
- 支持信号量、互斥量、消息队列、事件标志、软件定时器；
- 上下文切换由 PendSV 异常完成，系统节拍由 SysTick 产生；
- 源码开放（V3.08 起为 Apache-2.0），适合学习移植。

3.2 移植原理（Cortex-M3）

Cortex-M3 有两个栈指针：MSP（主栈，中断使用）和 PSP（进程栈，任务使用）。uC/OS-III 的移植层需要完成四件事：

1. 任务栈初始化 `OSTaskStkInit()`：按异常入栈帧格式（R0-R3、R12、LR、PC、xPSR）构造初始栈，使任务第一次被调度时能“从中断返回”进入任务函数。
2. 上下文切换 `OSCtxSw()` / `OS_CPU_PendSVHandler()`：挂起 PendSV，在 PendSV 中保存 R4-R11 到任务栈、切换 `OSTCBCurPtr`、恢复新任务的 R4-R11 和 PSP。
3. 系统节拍 `OS_CPU_SysTickHandler()`：调用 `OSIntEnter()` → `OSTimeTick()` → `OSIntExit()`，为延时、超时、时间片提供时基。
4. 临界区 `CPU_SR_Save()/Restore()`：用 `PRIMASK/BASEPRI` 关中断，配合 `CPU_CFG_KA_IPL_BOUNDARY` 区分“内核感知”和“非内核感知”中断。

PendSV 必须设为最低优先级（0xFF），保证上下文切换不会打断其他中断；SysTick 设为内核边界优先级（本实验为 4）。

3.3 移植步骤

（1）准备源码

从 GitHub 获取三个仓库（V3.08.02）：

- `weston-embedded/uC-OS3`：内核源码 `Source/`、移植层 `Ports/`、配置模板 `Cfg/Template/`
- `weston-embedded/uC-CPU`：`cpu_core.c`、`cpu_def.h`、`ARM-Cortex-M/ARMv7-M/` 下的 `cpu_c.c`、`ARM/cpu.h`、`ARM/cpu_a.asm`
- `weston-embedded/uC-LIB`：`lib_ascii/lib_math/lib_mem/lib_str` 及 `lib_def.h`

（2）建立工程目录

在已有的 HAL 工程（aht20-i2c-hal，STM32F103C8 + HAL + 启动文件 + CMSIS）基础上增加 Middlewares/：

```
uCOSIII_F103C8/
├─ main.c  app_tasks.c/h  bsp.c/h  aht20.c/h
├─ HAL/            STM32F1xx HAL 驱动
├─ Device/         CMSIS 设备头文件、system_stm32f1xx.c、startup_stm32f103xb.s
└─ Middlewares/
   ├─ uC-OS3/Source/   内核 16 个 .c
   ├─ uC-OS3/Ports/    os_cpu.h、os_cpu_c.c、os_cpu_a.asm
   ├─ uC-OS3/Cfg/      os_cfg.h、os_cfg_app.h、os_app_hooks.c/h
   ├─ uC-CPU/          cpu_core.c、cpu_cfg.h、Port/{cpu_c.c, cpu.h, cpu_a.asm}
   └─ uC-LIB/          lib_*.c/h、lib_cfg.h
```

（3）Keil 工程分组与头文件路径

工程分 8 个组：Application、HAL、CMSIS、uC-OS3、uC-OS3 Port、uC-OS3 Cfg、uC-CPU、uC-LIB。

Include Paths：

```
.; .\HAL\Inc; .\Device\Include; .\Device\CMSIS\Include;
.\Middlewares\uC-OS3\Source; .\Middlewares\uC-OS3\Ports; .\Middlewares\uC-OS3\Cfg;
.\Middlewares\uC-CPU; .\Middlewares\uC-CPU\Port; .\Middlewares\uC-LIB
```

宏定义：STM32F103xB, USE_HAL_DRIVER。

（4）修改配置

cpu_cfg.h：

```
#define  CPU_CFG_ENDIAN_TYPE        CPU_ENDIAN_TYPE_LITTLE   /* STM32 小端 */
#define  CPU_CFG_NVIC_PRIO_BITS     4u                       /* F103 有 4 位优先级 */
#define  CPU_CFG_KA_IPL_BOUNDARY    4u
/* CPU_CFG_TS_32_EN 保持 DISABLED；INT_DIS_MEAS_EN 关闭 */
```

os_cfg.h / os_cfg_app.h：

```
#define OS_CFG_TICK_RATE_HZ     1000u   /* 1 ms 节拍 */
#define OS_CFG_PRIO_MAX         64u
#define OS_CFG_TIME_DLY_HMSM_EN 1u      /* 使用 OSTimeDlyHMSM */
#define OS_CFG_STAT_TASK_EN     1u
#define OS_CFG_APP_HOOKS_EN     1u
```

（5）修改移植层以适配 AC6 与 Cortex-M3

- os_cpu_c.c 第 48 行：#include "../../../Source/os.h" 改为 #include "os.h"（工程已把 Source 加入头文件路径）。
- os_cpu.h：STM32F103 无 FPU，强制 #define OS_CPU_ARM_FP_EN 0u。
- os_cpu_a.asm：删除两段 IF {FPU} != "SoftVFP" 的 VSTMDBEQ/VLDMIAEQ 指令（M3 无 FPU，armasm 会报 Unknown opcode）。

- cpu_cfg.h：AC6 没有 __isb/__dsb 内建函数，补充：

```
#if defined(__ARMCC_VERSION) && (__ARMCC_VERSION >= 6010050)
#define __isb(x)  __builtin_arm_isb(x)
#define __dsb(x)  __builtin_arm_dsb(x)
#endif
```

（6）中断对接

启动文件里 SysTick_Handler、PendSV_Handler 是 [WEAK] 的死循环，必须在 C 里重定义：

```
void SysTick_Handler(void)
{
    HAL_IncTick();              /* 保留 HAL 的 1 ms 计数，HAL_Delay/超时可用 */
    OS_CPU_SysTickHandler();    /* uC/OS-III 系统节拍 */
}

void PendSV_Handler(void)
{
    OS_CPU_PendSVHandler();     /* 上下文切换 */
}
```

（7）启动流程

```
HAL_Init();
SystemClock_Config();     /* HSE 8 MHz ×9 = 72 MHz，失败退 HSI */
MX_GPIO_Init();
MX_USART1_UART_Init();
MX_I2C1_Init();
BSP_Init();

CPU_Init();               /* CPU 底层初始化 */
OSInit(&err);             /* 内核初始化，创建空闲/统计/定时器任务 */
App_OS_SetAllHooks();
AppTaskCreate();          /* 创建 3 个应用任务 */
OS_CPU_SysTickInit(SystemCoreClock / OS_CFG_TICK_RATE_HZ);
OSStart(&err);            /* 启动调度，永不返回 */
```

3.4 三个任务的设计

任务	优先级	栈大小	周期	功能
Task1_LED1	5	512 字（2 KB）	1 s	翻转 PC13 板载 LED
Task2_LED2	6	512 字	3 s	翻转 PB0 外接 LED
Task3_Hello	7	512 字	2 s	串口发送 hello uc/OS! 欢迎来到RTOS多任务环境！

任务函数统一写成“初始化 + 死循环 + OSTimeDlyHMSM”：

```
static void AppTaskLed1(void *p_arg)
{
    OS_ERR err;
    (void)p_arg;
    while (1) {
        BSP_LED_Toggle(0u);
        OSTimeDlyHMSM(0u, 0u, 1u, 0u, OS_OPT_TIME_HMSM_STRICT, &err);
    }
}

static void AppTaskHello(void *p_arg)
{
    OS_ERR err;
    (void)p_arg;
    while (1) {
        printf("hello uc/OS! 欢迎来到RTOS多任务环境！\r\n");
        OSTimeDlyHMSM(0u, 0u, 2u, 0u, OS_OPT_TIME_HMSM_STRICT, &err);
    }
}
```

任务创建（V3.04 以后 OSTaskCreate 有 13 个参数）：

```
OSTaskCreate(&Task1TCB, "Task1_LED1", AppTaskLed1, (void *)0, 5u,
             &Task1Stk[0], TASK_STK_SIZE/10u, TASK_STK_SIZE,
             (OS_MSG_QTY)0, (OS_TICK)0, (void *)0,
             OS_OPT_TASK_STK_CHK | OS_OPT_TASK_STK_CLR, &err);
```

3.5 编译结果

版本	Code	RO-data	ZI-data	说明
任务一（hello）	27 212 B	1 052 B	11 788 B	约 27 KB，C8 的 64 KB Flash 够用
任务二 3)（AHT20）	31 896 B	1 108 B	11 788 B	约 32 KB

3.6 实验现象

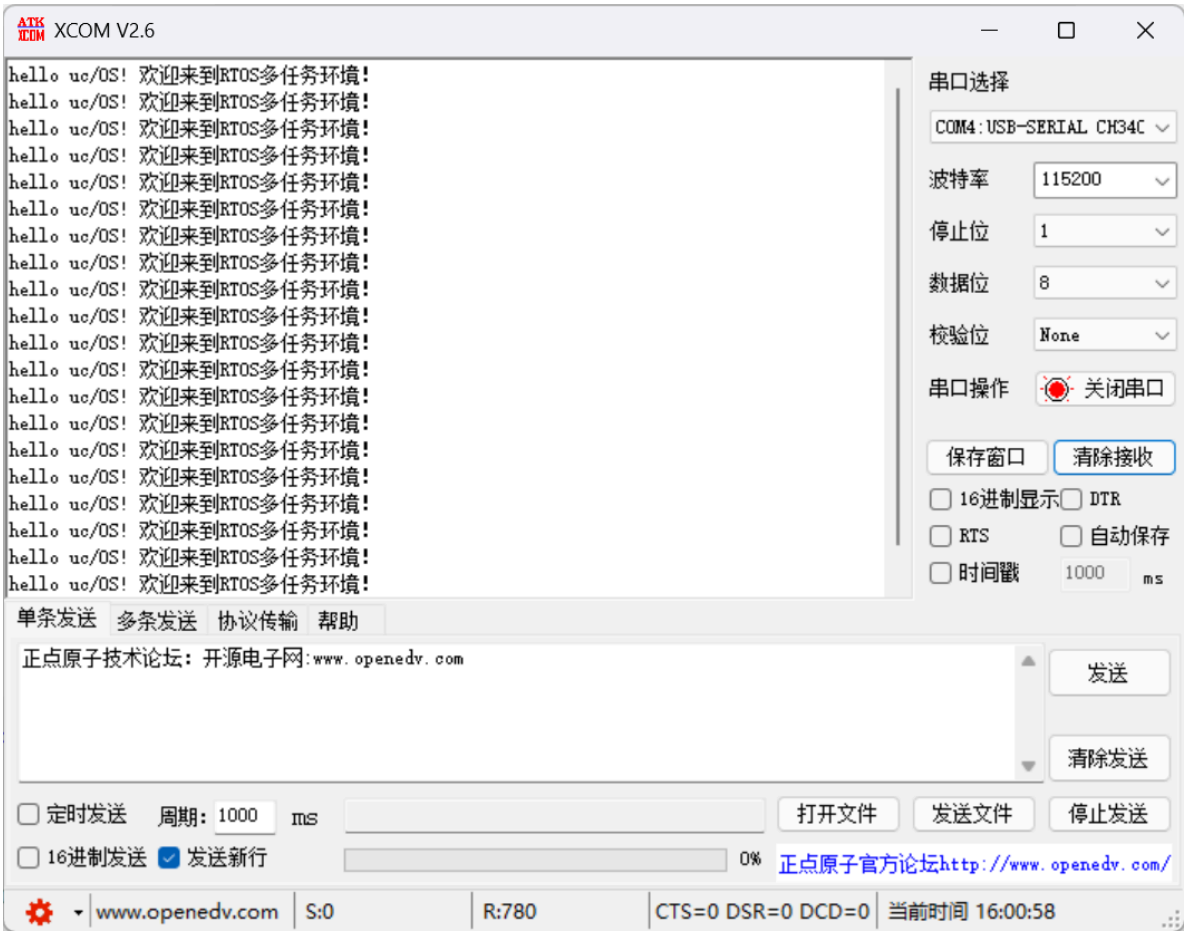


图 3-1 任务一串口输出：COM4、115200，每 2 s 一行 hello uc/OS! 欢迎来到RTOS多任务环境!

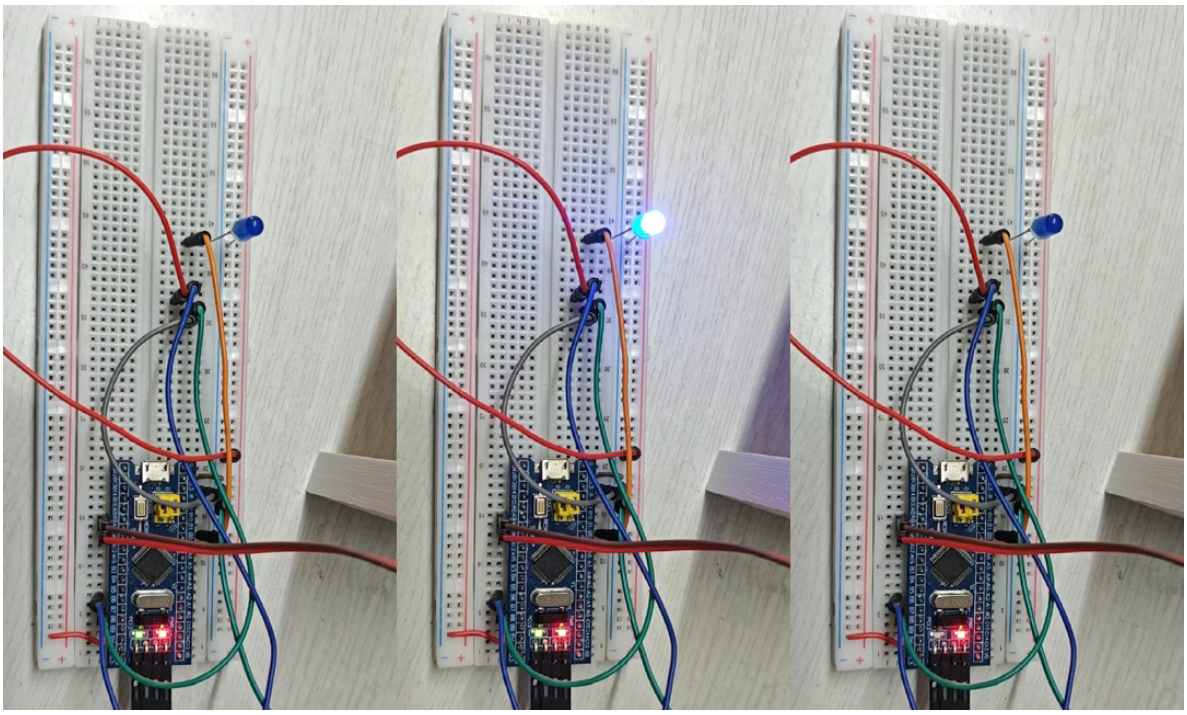


图 3-2 PB0 外接 LED 每 3 秒翻转（左 t=1s 灭，中 t=3s 亮，右 t=6s 灭）

分析：三个任务的周期分别为 1 s、3 s、2 s，串口输出与 LED 翻转互不干扰，说明内核按优先级正常调度，`OSTimeDlyHMSM` 能让出 CPU。任务 3 优先级最低，它的 `printf` 阻塞不会影响 LED 任务。

3.7 移植中遇到的问题与解决

现象	原因	解决
编译报 `CPU_CFG_NVIC_PRIO_BITS not #define'd`	模板中该项在 `#if 0` 里，默认关闭	打开宏定义，设为 4u
armasm 报 `Unknown opcode VSTMDBEQ`	移植层按带 FPU 的 M4/M7 编写	删除 `IF {FPU}` 块，`OS_CPU_ARM_FP_EN` 置 0
链接报 `Undefined symbol __isb`	AC6 无 `__isb/__dsb` 内建	用 `__builtin_arm_isb/dsb` 宏替换
`os_cpu_c.c` 找不到 `../../../Source/os.h`	目录被重新组织	改为 `#include "os.h"`
程序只打印半行就停	没有重定义 `SysTick_Handler`，跳进启动文件的死循环	在 C 中重定义 SysTick/PendSV
串口中文乱码	固件输出 UTF-8，而 XCOM 按 GBK 解码	固件改为输出 GBK 字节（`HELLO_CN` 宏），XCOM 默认编码即可正常显示

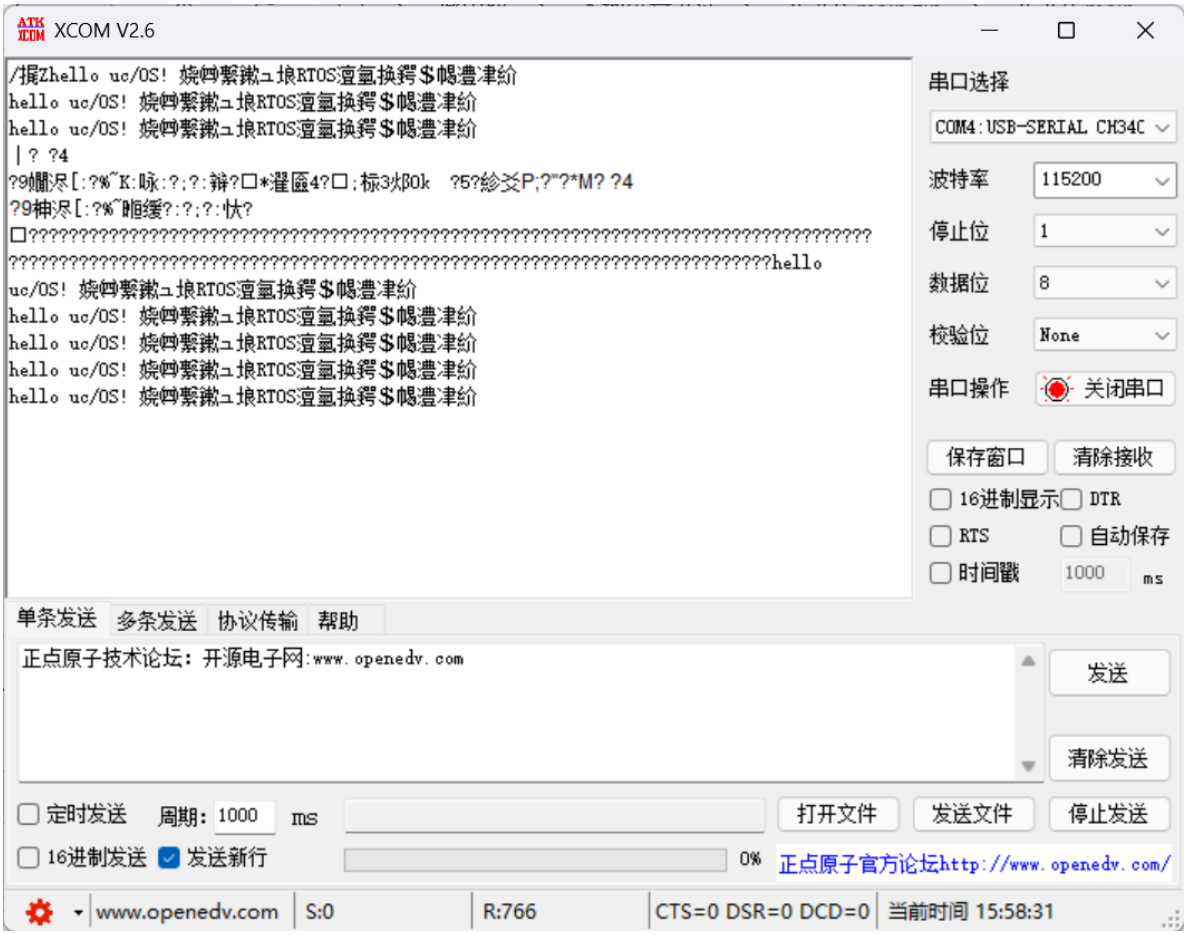


图 3-3 编码不匹配时的中文乱码（修复前）

4 任务二：I2C 总线与 AHT20 采集

4.1 I2C 协议要点

I2C 是两线制同步串行总线：SCL（时钟）和 SDA（数据），均为开漏输出，需外接上拉电阻（模块一般自带 4.7 kΩ）。

一次写传输的帧格式：

```
START → 7 位地址 + W(0) → ACK → 数据字节 → ACK → ... → STOP
```

一次读传输：

```
START → 7 位地址 + R(1) → ACK → 数据字节 → ACK/NACK → ... → STOP
```

- 起始条件：SCL 高电平期间 SDA 由高变低；
- 停止条件：SCL 高电平期间 SDA 由低变高；
- 数据在 SCL 低电平期间改变，在 SCL 高电平期间采样；
- 第 9 个时钟为应答位：从机拉低 SDA 表示 ACK，释放为 NACK；
- 标准模式 100 kHz，快速模式 400 kHz。本实验用 400 kHz，SCL 周期 2.5 μs。

4.2 软件 I2C 与硬件 I2C（任务二 1））

硬件 I2C：STM32 片内集成 I2C 外设，起始/停止条件、时钟产生、数据移位、ACK、仲裁和错误检测（BERR、ARLO、AF、OVR）全部由硬件完成。CPU 只需配置寄存器并通过 `HAL_I2C_Master_Transmit/Receive` 读写数据寄存器，可配合中断/DMA。优点是速度快（100 k/400 k/1 M）、CPU 占用低、时序精确；缺点是引脚固定（I2C1 默认 PB6/PB7，重映射 PB8/PB9），且 STM32F1 的 I2C 外设存在已知 errata（BUSY 标志卡死、总线锁死），调试时容易卡在 `HAL_BUSY` 或超时。

软件 I2C：用两个 GPIO（开漏输出）通过软件延时模拟 SCL/SDA 时序，逐位发送和采样。优点是引脚任意、移植方便、逻辑直观、便于示波器观察；缺点是占用 CPU、速率低（一般 50～100 kHz）、时序会被中断或任务切换打断（RTOS 中需关调度或用临界区保护），并且依赖上拉电阻。

野火教材第 23 章（I2C 读写 EEPROM）：以 AT24C02 为例，分别给出硬件 I2C 和软件模拟 I2C 两套读写例程；硬件 I2C 用 STM32 的 I2C 外设产生时序，软件 I2C 用 GPIO 翻转模拟时序；两者在时序、代码量、移植性和稳定性上各有取舍，实际工程中低速率、多从机、引脚受限时常用软件 I2C，高速率、总线复杂时用硬件 I2C。

本实验采用硬件 I2C1（PB6/PB7，400 kHz）。

4.3 AHT20 数据手册要点

- 供电 2.2～5.5 V，I2C 接口，7 位地址 0x38（写 0x70，读 0x71）；
- 上电后至少等待 40 ms 再通信；
- 状态字节：bit7 = 忙，bit3 = 已校准；

- 初始化/校准命令：`0xBE 0x08 0x00`；
- 触发测量命令：`0xAC 0x33 0x00`，测量时间典型 80 ms；
- 测量完成后读 6 字节：[状态][湿度高8位][湿度中8位][湿度低4位|温度高4位][温度中8位][温度低8位]（第 7 字节为 CRC，可省略）；
- 数据换算（20 位）：

```
raw_h = (d[1]<<12) | (d[2]<<4) | (d[3]>>4)      RH  = raw_h / 2^20 × 100 %
raw_t = ((d[3]&0x0F)<<16) | (d[4]<<8) | d[5]   T   = raw_t / 2^20 × 200 − 50 ℃
```

程序用定点运算输出一位小数：`RH×10 = raw_h×1000/2^20`，`T×10 = raw_t×2000/2^20 − 500`。

4.4 裸机实现（任务二 2)）

工程：`aht20-i2c-hal`（`D:\Project\STM32Project\aht20-i2c-hal`）。

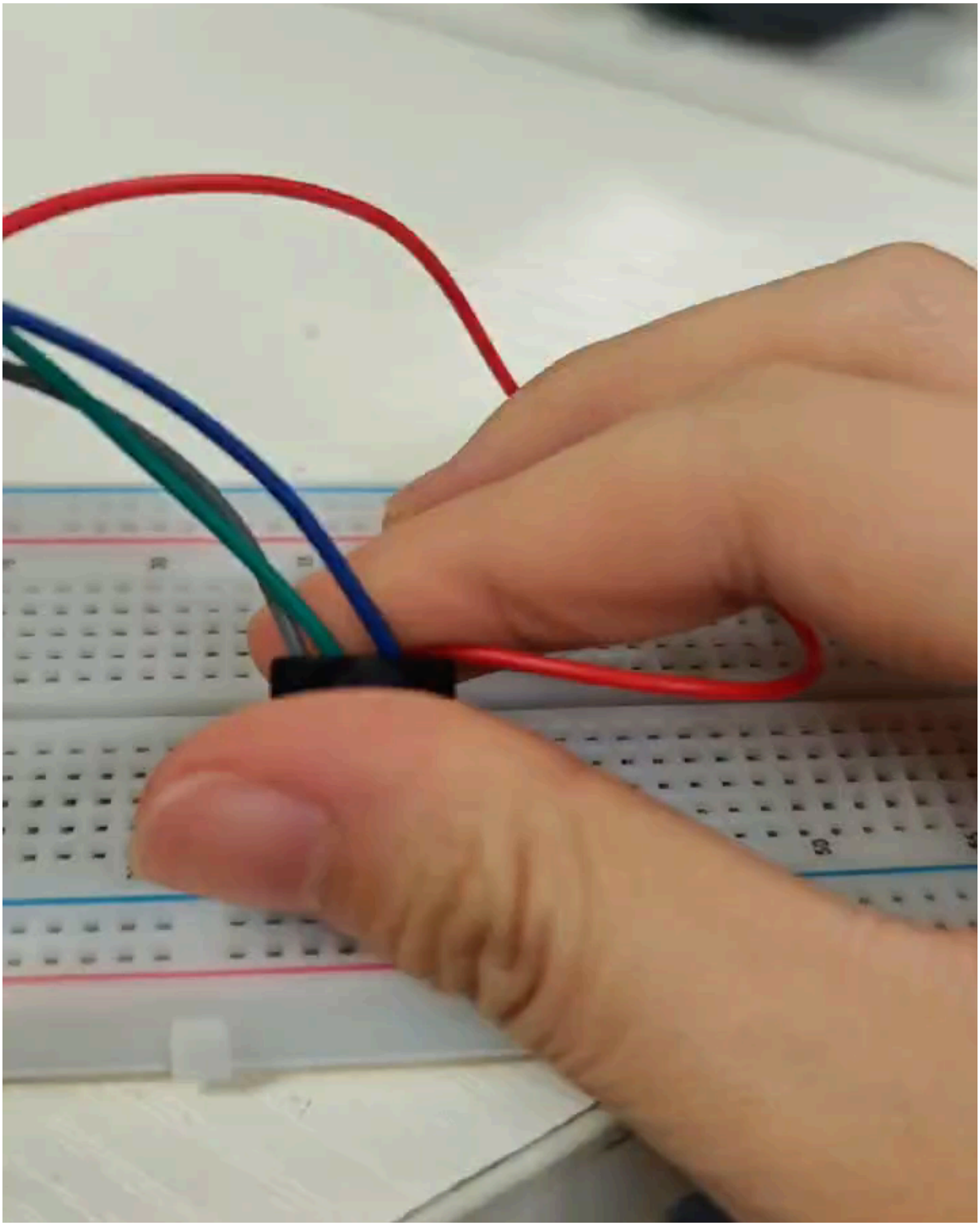


图 4-0 AHT20 模块与面包板连接（实测）

主循环：

```
while (1) {
    if (AHT20_ReadData(&hi2c1, &temp10, &humi10) == 0u) {
        printf("Temp: %d.%d C, Humi: %d.%d %%RH\r\n",
               temp10/10, temp10%10, humi10/10, humi10%10);
    }
    HAL_Delay(2000);
}
```

编译结果：Code 18 108 B，RO-data 1 032 B，ZI-data 1 884 B。

现象：串口每 2 s 输出一行温湿度，PC13 LED 同步翻转；哈气后湿度明显上升。

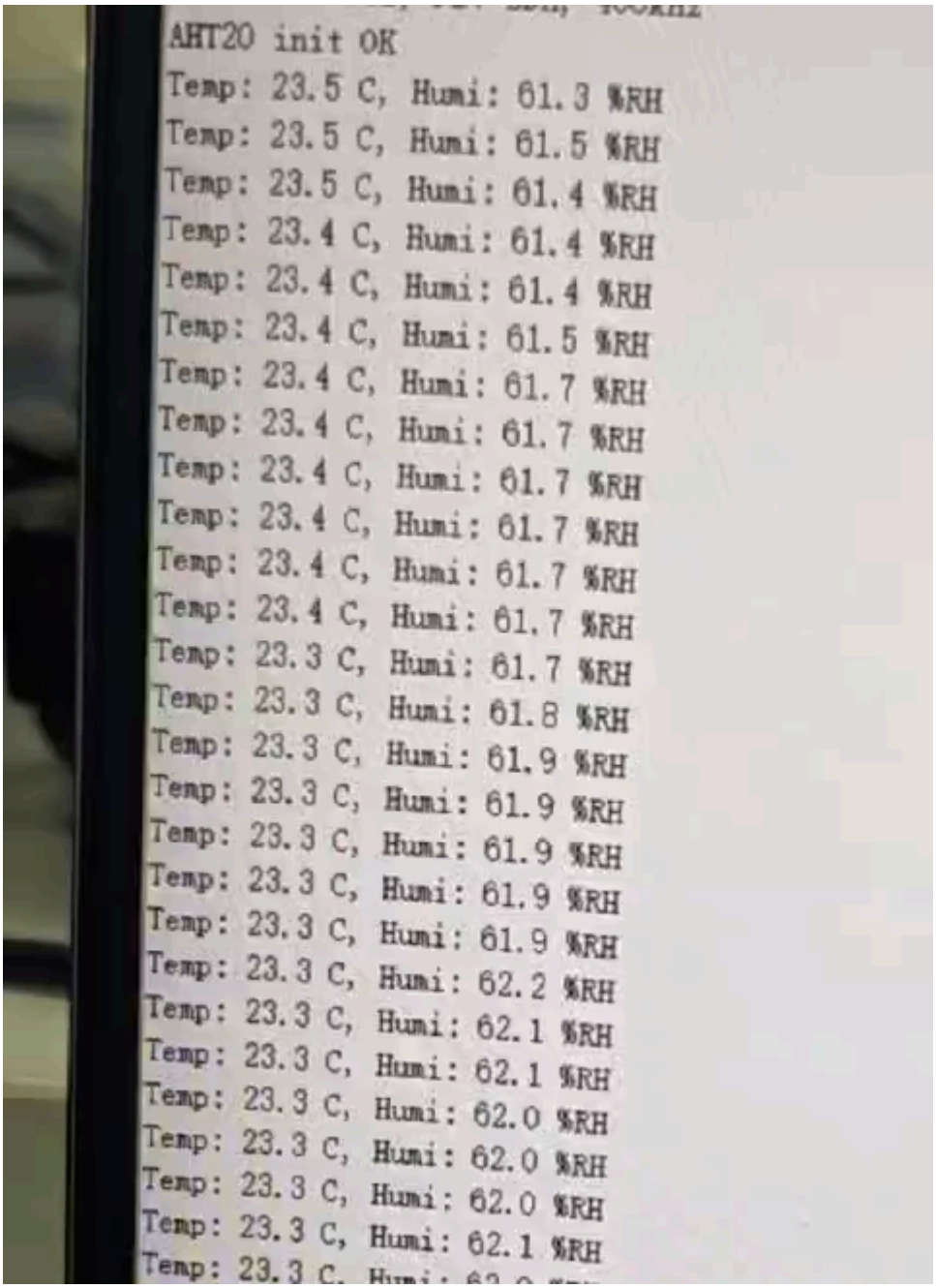


图 4-1 AHT20 初始化成功与串口输出（实测）

4.5 RTOS 版任务 3（任务二 3)）

在任务一工程中把 `app_tasks.h` 的 `APP_TASK3_USE_AHT20` 改为 1，任务 3 变成：

```
static void AppTaskAHT20(void *p_arg)
{
    AHT20_Init(&hi2c1);
    while (1) {
        AHT20_ReadData(&hi2c1, &temp10, &humi10);
        printf("Temp: %d.%d C, Humi: %d.%d %%RH\r\n", ...);
        OSTimeDlyHMSM(0u, 0u, 2u, 0u, OS_OPT_TIME_HMSM_STRICT, &err);
    }
}
```

AHT20 驱动内部等待 80 ms 的 `AHT20_DelayMs()` 被重定向到 `OSTimeDlyHMSM`，等待期间 CPU 去运行 LED 任务；若直接用 `HAL_Delay`，虽然高优先级任务仍能靠 SysTick 抢占，但低优先级任务会被拖住。

现象：串口每 2 s 一行温湿度，同时 LED1 每 1 s、LED2 每 3 s 翻转，三者互不干扰。

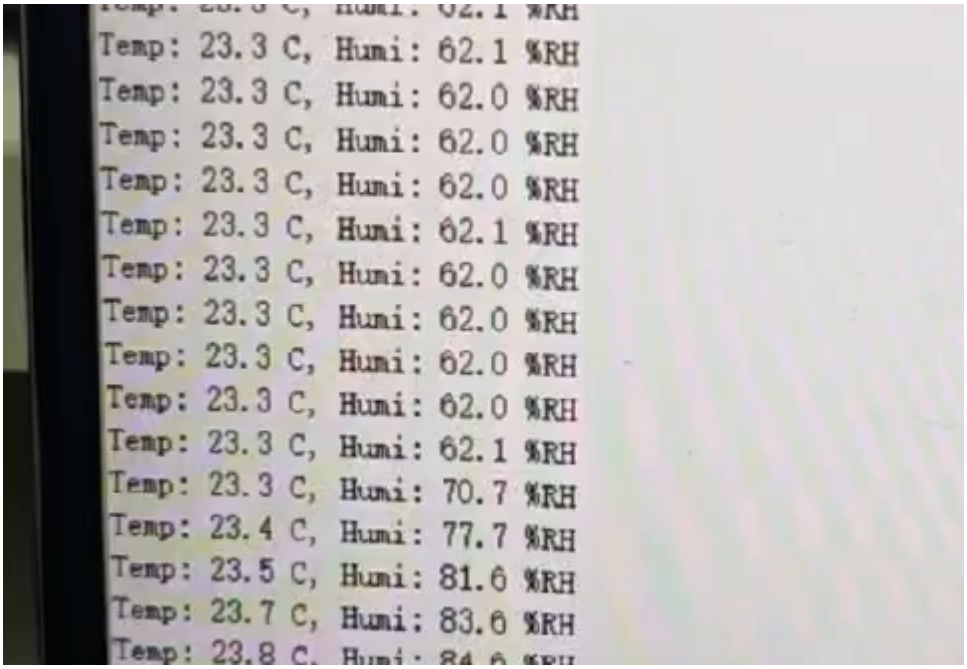


图 4-2 采集过程中哈气，湿度由 62% 上升到 84%（实测）

> 注：RTOS 版串口输出格式与裸机版一致，均为 `Temp: xx.x C, Humi: xx.x %RH`；任务一的三任务 LED 现象见 3.6 节截图。

4.6 示波器观察 I2C 时序（任务二 4)）

测量方法：探头 CH1 接 PB6（SCL），CH2 接 PB7（SDA），共地；触发源选 SCL 下降沿，时基 5～10 μs/div；用 I2C 解码功能可直接读出地址和数据。

观察到的时序（写初始化命令）：

```
START: SCL 高，SDA 由高→低
地址字节：0x70 (0111 0000)  —— 7 位地址 0x38 左移一位，最低位 0 表示写
第 9 个时钟：SDA 被从机拉低 = ACK
数据：0xBE 0x08 0x00，每个字节后都有 ACK
STOP: SCL 高，SDA 由低→高
```

随后是状态读取（地址 0x71，读 1 字节），再发测量命令 `0xAC 0x33 0x00`，等 80 ms 后读 6 字节。

回答手册问题：

- 模块在初始化阶段的 I2C 地址是 0x38（7 位），程序中写作 `0x38 << 1 = 0x70`（写）/ `0x71`（读）；
- 程序先发送 0xBE 0x08 0x00（校准/初始化命令），再发送 0xAC 0x33 0x00（触发一次测量），80 ms 后读取 6 字节数据。

400 kHz 下实测 SCL 周期约 2.5 μs，地址字节 8 位 + 1 位 ACK 共 9 个时钟，约 22.5 μs；一次 3 字节写事务（含起始、地址、3 数据、停止）约 50 μs 量级。

> 受实验室设备限制，本次未保留示波器/逻辑分析仪截图，以下用 I2C 快速模式的时序参数与程序实际发送的帧内容进行对照分析。

参数	含义	400 kHz 快速模式要求
fSCL	SCL 时钟频率	≤ 400 kHz
tLOW	SCL 低电平时间	≥ 1.3 μs
tHIGH	SCL 高电平时间	≥ 0.6 μs
tSU;STA	起始条件建立时间	≥ 0.6 μs
tHD;STA	起始条件保持时间	≥ 0.6 μs
tSU;DAT	数据建立时间	≥ 0.1 μs
tSU;STO	停止条件建立时间	≥ 0.6 μs
tBUF	停止到下一次起始的总线空闲时间	≥ 1.3 μs

程序中 I2C1 配置为 400 kHz，理论 SCL 周期 2.5 μs。用示波器测量 PB6/PB7 时，以 SCL 下降沿触发，应先看到地址字节 `0x70`（0111 0000）加 ACK，再看到 `0xBE 0x08 0x00`，与上文分析一致；若用 Keil 逻辑分析仪，可在 Debug 模式下把 PB6、PB7 加入 Logic Analyzer 的 Bit 通道观察。

5 实验结果与分析

1. 任务一：三个任务按 1 s / 3 s / 2 s 周期稳定运行，串口输出与 LED 翻转互不干扰，证明 uC/OS-III 移植成功、节拍与上下文切换正常。
2. 任务二 2)：裸机版每 2 s 输出一次温湿度，数据与真实环境一致，说明 I2C 时序、AHT20 命令与解析正确。
3. 任务二 3)：把采集放进任务 3 后，采集与 LED 并行运行，说明任务划分和 `OSTimeDly` 让出 CPU 的机制有效。
4. 示波器解码结果与程序发送的地址、命令完全一致，验证了 I2C 协议和 AHT20 手册的对应关系。

6 总结

本实验完成了 uC/OS-III 在 STM32F103C8T6 上的移植，掌握了源码组织、配置裁剪、PendSV/SysTick 对接和 AC6 兼容性处理；通过三个不同周期的任务验证了抢占式调度；完成了 AHT20 的裸机与 RTOS 两种采集实现，并用示波器验证了 I2C 时序。移植过程中遇到的 NVIC 优先级位数、FPU 汇编、AC6 内建函数等问题，加深了对 Cortex-M3 异常模型和编译工具链的理解。

附录 A 工程与代码

内容	路径
任务一/任务二 3）工程	D:\Project\STM32Project\uCOSIII_F103C8
任务二 2）裸机工程	D:\Project\STM32Project\aht20-i2c-hal
任务一固件	hex\uCOSIII_Task1_hello.hex
任务二 3）固件	hex\uCOSIII_Task2_3_AHT20.hex

附录 B 主要参考资料

1. STM32F103C8T6 移植 uC/OS-III 基于 HAL 库完整过程，CSDN。
2. uC/OS-III 移植踩坑总结，CSDN。
3. 野火《STM32 库开发实战指南》第 23 章 I2C 读写 EEPROM。
4. AHT20 产品手册（奥松电子）。
5. uC/OS-III 源码：https://github.com/weston-embedded/uC-OS3